

ML-BASED KPI PREDICTION xAPP

FOR O-RAN NETWORK MONITORING

COMPLETE TECHNICAL & VIVA GUIDE

Contents

1. Project Overview.....	5
1.1 What is O-RAN?.....	5
1.2 What is a KPI here?	5
1.3 How the Project is Structured	5
2. The Five Training Scripts — What Each One Does.....	6
Script 1 — clean_dataset.py	6
Input	6
What it does step by step	6
Outputs.....	7
Script 2 — label_dataset.py	7
Input	7
How the label is created — quantile scoring	7
Output	8
Script 3 — train_random_forest.py	8
Input	8
What happens inside	8
Key parameters	9
Output	9
Script 4 — train_xgboost.py	9
Input	9
What happens inside — same steps as RF, different model settings	9
Outputs.....	9
Script 5 — train_lstm.py	10
Why the original script crashed	10
How the generator works.....	10
Key parameters	10
LSTM Architecture.....	10
Outputs.....	11
3. The Three ML Models — How Each One Works.....	12
3.1 Random Forest	12
The simple explanation.....	12
Technically	12
Why it works well	12
Strengths for this project.....	12
Results on your dataset	12

Feature importance (top 5 most important KPIs)	12
3.2 XGBoost	13
The simple explanation	13
Technically	13
Why it outperforms Random Forest	13
Results on your dataset	13
3.3 LSTM	14
The simple explanation	14
Technically	14
Why it scored lower on your dataset	14
Results on your dataset	14
3.4 Side-by-side comparison	14
4. How the App Works — Live Monitoring Explained	16
4.1 What the live monitoring is	16
4.2 The complete data flow — step by step	16
4.3 The stream state — how the server remembers its position	16
4.4 What each element on the live monitoring screen shows	17
4.5 The manual prediction dashboard (home page /)	17
Preset buttons	17
Model selector	18
Predict button flow	18
5. API Parameters — Every Endpoint Explained	19
5.1 POST /api/predict	19
Query parameter	19
Request body — all 18 fields required	19
Response fields	20
Error codes	20
5.2 GET /api/stream/next	20
5.3 GET /api/health	21
5.4 GET /api/comparison	21
5.5 GET /api/stream/reset	21
5.6 GET /api/stream/status	21
6. Understanding the Evaluation Metrics	22
6.1 The Confusion Matrix	22
6.2 The Four Metrics Explained Simply	22
6.3 Why all three models have 100% Precision	22

6.4 Three-way results summary	23
7. Key Viva Questions — with Suggested Answers.....	24
Q: Does this project connect to a real 5G network?	24
Q: Why did clean_dataset.py not drop columns like IMSI and slice_id in the multi-file version?	24
Q: Why did the original LSTM crash with a memory error?.....	24
Q: Why is LSTM's recall so low with 19 million rows?	24
Q: What is the difference between actual_risk and predicted_risk in the stream endpoint?	24
Q: What would you do differently with more time?	24
8. Quick Reference — Numbers to Know for Your Viva.....	26

1. Project Overview

This project is a web dashboard — called an xApp — that monitors a 5G radio network and predicts whether the network is degrading. It was built as a Final Year Project using real KPI (Key Performance Indicator) data collected from an O-RAN software-defined radio testbed.

Instead of a network engineer manually watching dozens of numbers on a screen, this system uses three Machine Learning models to do it automatically. Every two seconds, a new row of KPI measurements arrives, the model analyses it, and the dashboard updates with a verdict: Normal or Degraded.

1.1 What is O-RAN?

O-RAN stands for Open Radio Access Network. Traditional 5G equipment is proprietary — all hardware and software must come from one vendor. O-RAN breaks this by using open standards, allowing different vendors's hardware and software to work together.

The key component relevant to this project is the Near-RT RIC (RAN Intelligent Controller), which hosts small applications called xApps that can monitor the network and influence decisions in near real-time. This project builds the ML and dashboard components of such an xApp.

1.2 What is a KPI here?

A KPI is a radio performance measurement. The base station records 18 of them every 250 milliseconds. Examples:

- `dl_cqi` — how good the signal quality is, rated 0–15 by the phone itself. Low = bad.
- `rx_errors_uplink_pct` — percentage of packets from the phone that were corrupted. High = bad.
- `tx_brate_downlink_mbps` — actual download speed the user is getting. Low = bad.
- `dl_mcs` — which encoding scheme the base station chose. Low = struggling with the channel.

1.3 How the Project is Structured

The project has three layers:

- Data Pipeline — five Python scripts that clean, label, and train models from the raw CSV files.
- FastAPI Backend — a Python web server that loads the trained models and serves predictions via a REST API.
- Next.js Frontend — a web dashboard with two pages: manual prediction and live monitoring.

2. The Five Training Scripts — What Each One Does

These scripts must be run in order, from the backend/ folder with the virtual environment active. Each one depends on the output of the previous one.

```
python app\ml\clean_dataset.py
python app\ml\label_dataset.py
python app\ml\train_random_forest.py
python app\ml\train_xgboost.py
python app\ml\train_lstm.py
```

Script 1 — clean_dataset.py

This is the first script that must run. It reads all the raw CSV files, validates them, merges them together, cleans the combined data, and saves two output files.

Input

All .csv files inside: backend/data/raw/

You can have one file or dozens — the script finds all of them automatically.

What it does step by step

Step	Action	Why
1	Scan data/raw/ for all CSV files	Finds every recording session automatically
2	Validate each file	Skips empty, corrupted, or column-mismatched files with a warning
3	Merge all valid files	Stacks all rows into one combined DataFrame
4	Save combined_dataset.csv	Checkpoint — raw merged data before any cleaning
5	Drop completely empty columns	Removes Unnamed separator columns (artefacts of CSV export)
5b	Drop known identity/config columns	Removes IMSI, RNTI, num_ues, slice_id etc — these are experiment metadata, not radio performance
6	Drop zero-variance columns	Removes any remaining columns with only one unique value
7	Remove duplicate rows	Removes rows that appear in more than one recording session
8	Rename columns to snake_case	e.g. "rx_errors uplink (%)" becomes "rx_errors_uplink_pct"
9	Convert timestamp to datetime	Converts Unix milliseconds to a proper date/time object

Step	Action	Why
10	Fill missing values with medians	Uses median (not mean) because it is not affected by outliers
11	Validate ranges	Checks for negatives, CQI outside 0–15, MCS outside 0–28
12	Create prb_grant_ratio feature	Derives a congestion indicator: $\text{granted_prbs} / (\text{requested_prbs} + 1)$
13	Sort by timestamp	Ensures rows are in chronological order for LSTM
14	Print summary table	Shows files found/skipped, rows, columns, missing values
15	Save cleaned_dataset.csv	Clean output ready for the next script

Outputs

- backend/data/processed/combined_dataset.csv — raw merged data (checkpoint)
- backend/data/processed/cleaned_dataset.csv — fully cleaned, ready for labelling

Why we never remove rows: zero-throughput rows mean the UE was idle (normal). 100%-error rows are the degradation events we want to detect. Removing them would make the model blind to what we are trying to predict.

Script 2 — label_dataset.py

This script reads the cleaned dataset and creates a target label column called `degradation_risk`. This label is what the ML models learn to predict. It did not exist in the raw data — we create it using domain knowledge about what "bad" radio performance looks like.

Input

backend/data/processed/cleaned_dataset.csv

How the label is created — quantile scoring

Each row is checked against 7 conditions. Each condition that is "bad" adds 1 point to a degradation score. A row scoring 2 or more is labelled Degraded (1). Below 2 is Normal (0).

#	Feature	Direction	Threshold	Meaning
1	rx_errors_uplink_pct	High = Bad	Above Q90 (57.1%)	Over half the UE's packets corrupted — retransmissions waste bandwidth
2	dl_cqi	Low = Bad	Below Q25 (6.33)	UE reports poor channel quality — base station uses slower modulation
3	tx_brake_downlink_mbps	Low = Bad	Below Q10 (0.003 Mbps)	Near-zero throughput — user is getting almost no data

#	Feature	Direction	Threshold	Meaning
4	prb_grant_ratio	Low = Bad	Below Q25 (0.15)	Scheduler granting only 15% of requested bandwidth — heavy congestion
5	dl_mcs	Low = Bad	Below Q25 (6.57)	Base station using low/simple modulation — struggling with the link
6	ul_sinr	Low = Bad	Below Q25 of active rows	Weak signal vs noise when UE is actually transmitting
7	ul_turbo_iters	High = Bad	Above Q75 of active rows	Many decoder attempts needed — packets are heavily damaged

Why quantile thresholds and not hardcoded numbers? A hardcoded threshold like "error > 30%" is arbitrary — it might be wrong for a different dataset. Quantiles say "flag the worst 10% of THIS dataset", so the thresholds automatically adapt to however many files you add.

Why require 2 or more bad conditions? One bad reading can have innocent reasons — low throughput might just mean the UE has nothing to send. Two simultaneous bad KPIs is far stronger evidence of genuine degradation.

Output

- backend/data/processed/labeled_dataset.csv — 18 features + degradation_score + degradation_risk

Script 3 — train_random_forest.py

Trains the Random Forest classifier. This is the most explainable model in the project and the easiest to describe in a viva.

Input

backend/data/processed/labeled_dataset.csv

What happens inside

1. Load the labeled dataset and separate features (X) from label (y).
2. Drop identity columns (IMSI, RNTI, etc.) and check for infinity values — safety net.
3. Split 80% for training, 20% for testing. shuffle=False preserves time order.
4. Train RandomForestClassifier with 100 trees and class_weight="balanced".
5. Evaluate: accuracy, precision, recall, F1, confusion matrix.
6. Print feature importances — which KPIs matter most.
7. Save the trained model as random_forest_model.pkl.

Key parameters

Parameter	Value	What it means
n_estimators	100	Number of decision trees. More trees = more accurate but slower to train.
max_depth	None	Trees grow until all leaves are pure. The ensemble corrects for overfitting.
class_weight	"balanced"	Prevents the 69% Normal majority from dominating. Degraded events get 2.2× more influence.
shuffle	False	Preserves time order. Shuffling would leak future data into training.
random_state	42	Fixed seed so the same result is produced every run.
n_jobs	-1	Use all CPU cores to speed up training.

Output

- backend/models/random_forest_model.pkl — saved trained model

Script 4 — train_xgboost.py

Trains the XGBoost classifier. XGBoost consistently produces the highest accuracy on tabular data and is your best-performing model.

Input

backend/data/processed/labeled_dataset.csv

What happens inside — same steps as RF, different model settings

Parameter	Value	What it means
n_estimators	100	Number of boosting rounds. Same as RF for fair comparison.
max_depth	4	Trees are kept shallow. Boosting already corrects errors, deep trees would overfit.
learning_rate	0.1	How much each tree corrects the previous one. 0.1 is a standard safe default.
scale_pos_weight	~2.16	XGBoost equivalent of class_weight="balanced". Computed as Normal count / Degraded count.
eval_metric	"logloss"	The loss function for binary classification — standard choice.
verbosity	0	Suppress per-tree training output.

Outputs

- backend/models/xgboost_model.pkl — saved trained model

- backend/models/comparison_report.json — RF vs XGBoost side-by-side metrics

Script 5 — train_lstm.py

Trains the LSTM neural network. This script has been rewritten to use a memory-efficient generator so it can handle datasets of any size without running out of RAM.

Why the original script crashed

The original version tried to build all sliding-window sequences at once:

19 million rows × 20 window × 18 features × 4 bytes = 27 GB RAM required

No normal PC can allocate 27 GB for a single array. The new version uses a Keras generator that creates only one small batch at a time.

How the generator works

A generator is an object that feeds data to Keras one batch at a time during training. Instead of creating all 19 million windows upfront, it creates 512 windows, gives them to the model, then throws them away and creates the next 512. Memory usage stays around 750 KB per batch regardless of dataset size.

Key parameters

Parameter	Value	What it means
WINDOW_SIZE	20	How many past readings the LSTM sees per prediction. 20 rows × 250ms = 5 seconds of history.
EPOCHS	30	Maximum training rounds. Early stopping usually stops training before this.
BATCH_SIZE	512	Sequences per gradient update. Larger than RF/XGB because the dataset is much bigger.
TEST_SIZE	0.2	20% held back for testing — matches RF and XGBoost for fair comparison.
patience	5	Early stopping: if validation loss does not improve for 5 epochs, stop and revert to best weights.
class_weight Degraded	~2.38	Normal count / Degraded count — upweights the minority class.

LSTM Architecture

Layer	Parameters	What it does
LSTM(32)	6,528	32 memory cells process the 20-step sequence. Each cell remembers what is relevant and forgets what is not.

Layer	Parameters	What it does
Dropout(0.2)	0	Randomly zeros 20% of outputs during training. Forces the model to learn robust patterns, not memorise.
Dense(16, relu)	528	Fully connected layer. relu = $\max(0, x)$ introduces non-linearity so the model can learn complex patterns.
Dense(1, sigmoid)	17	Outputs a probability 0.0–1.0. Above 0.5 = Degraded. $\text{sigmoid}(x) = 1/(1+e^{-x})$.
Total	7,073	Very small network — appropriate for this dataset size.

Outputs

- backend/models/lstm_model.keras — saved LSTM network
- backend/models/lstm_scaler.pkl — MinMaxScaler used to scale features (must be saved alongside the model)
- backend/models/comparison_report.json — updated with all three models

Why save the scaler? At prediction time, new KPI values must be scaled the same way as the training data. Without the scaler file, the LSTM predictions would be meaningless.

3. The Three ML Models — How Each One Works

3.1 Random Forest

The simple explanation

Imagine asking 100 friends whether to bring an umbrella. Each friend checks different things — clouds, humidity, season, yesterday's weather. They all vote yes or no. You go with whatever most say. That is a Random Forest.

Technically

A Decision Tree is a flowchart of yes/no questions: "Is dl_cqi below 6.3? If yes → Is error rate above 57%? If yes → DEGRADED". One tree is unreliable — it overfits to the training data. A Random Forest builds 100 trees, each trained on a random subset of rows and features. All 100 vote, and majority wins.

Why it works well

100 imperfect trees working together are much more reliable than one supposedly perfect tree. Individual mistakes cancel out across the ensemble.

Strengths for this project

- Fast to train even on large datasets
- No feature scaling needed
- Produces feature importance ranking — invaluable for viva explanation
- Resistant to overfitting because 100 independent trees vote

Results on your dataset

Metric	Value
Accuracy	98.56%
Precision	100.00%
Recall	94.87%
F1 Score	97.37%
False Negatives (missed events)	6 of 117

Feature importance (top 5 most important KPIs)

Rank	Feature	Importance	Why it matters
1	dl_mcs	22.97%	The base station chooses MCS based on channel quality — it summarises everything

Rank	Feature	Importance	Why it matters
2	dl_cqi	17.44%	The phone's own report card on how good the signal is
3	ul_turbo_iters	10.08%	More decoder iterations = more damaged packets = weaker uplink
4	sum_granted_prbs	7.06%	Absolute amount of bandwidth granted
5	prb_grant_ratio	6.56%	Our engineered feature — congestion indicator — ranks 5th, proving it added value

3.2 XGBoost

The simple explanation

Imagine you are a student doing practice tests. After test 1, you focus on the questions you got wrong. After test 2, you focus on the remaining mistakes. You repeat this until you are getting almost everything right. Each attempt corrects the errors of the last. That is XGBoost.

Technically

XGBoost also uses decision trees, but builds them sequentially. Tree 1 makes initial predictions. Tree 2 is specifically built to correct the rows Tree 1 got wrong. Tree 3 corrects what Trees 1 and 2 still got wrong. The final prediction is the weighted sum of all 100 trees.

The "gradient" in gradient boosting refers to the mathematical technique used to figure out where the next tree should focus — it points in the direction that reduces errors most.

Why it outperforms Random Forest

Random Forest trees are independent — they do not learn from each other. XGBoost trees are sequential — each one is specifically designed to fix what came before. This is why XGBoost caught 4 more degradation events (missed only 2 vs RF's 6). On structured tabular data, XGBoost almost always wins.

Results on your dataset

Metric	Value
Accuracy	99.52%
Precision	100.00%
Recall	98.29%
F1 Score	99.14%
False Negatives (missed events)	2 of 117

3.3 LSTM

The simple explanation

Imagine monitoring a patient's blood pressure. If you see a single reading of 140, you might not be alarmed. But if you see 20 readings gradually climbing: 115 → 120 → 126 → 131 → 137 → 140... you know something is wrong. The trend tells the story, not the single number. LSTM sees the trend.

Technically

LSTM is a Recurrent Neural Network with memory cells. At each time step (each row), the memory cell decides what to remember from the past, what to forget, and what to output. After processing all 20 rows in the window, the final output goes to a Dense layer that produces the degradation probability.

Why it scored lower on your dataset

LSTM needs long sequences to learn temporal patterns — ideally hours or days of data. With a window of 20 rows at 250ms each, the LSTM sees only 5 seconds of history per prediction. With 19 million rows total, the individual sequences are still only 5 seconds long. The model cannot learn patterns like "CQI dropping over 30 seconds typically leads to degradation 10 seconds later."

This is the most honest and academically interesting result in the project. LSTM's low recall is expected and correct — it demonstrates that model selection must consider practical data constraints, not just theoretical suitability.

LSTM's low recall (18.8%) is not a bug. It means the model is very conservative — when uncertain, it predicts Normal rather than risk a false alarm. Precision is still 100%: every time it does say Degraded, it is correct.

Results on your dataset

Metric	Value	Interpretation
Accuracy	77.27%	Lower because it misses many degradation events
Precision	100.00%	Zero false alarms — every Degraded prediction was correct
Recall	18.80%	Only catches ~1 in 5 actual degradation events — needs more data
F1 Score	31.65%	Harmonic mean of Precision and Recall — reflects the recall problem
False Negatives	95 of 117	Missed 95 degradation events — the main weakness

3.4 Side-by-side comparison

	Random Forest	XGBoost	LSTM
How it thinks	100 trees vote in parallel	Trees fix each other's mistakes sequentially	Processes a sequence of 20 rows with memory
Input per prediction	1 row (18 features)	1 row (18 features)	20 rows ($20 \times 18 = 360$ values)
Needs feature scaling	No	No	Yes — MinMaxScaler to [0,1]
Training time (19M rows)	Fast (minutes)	Fast (minutes)	Slow (hours with generator)
F1 Score	97.37%	99.14%	31.65%
Missed events	6 of 117	2 of 117	95 of 117
Best when	Explainability matters	Best accuracy on tabular data	Long time-series (hours+) available

4. How the App Works — Live Monitoring Explained

4.1 What the live monitoring is

The live monitoring page (/live) simulates a real O-RAN monitoring feed by replaying the labeled CSV dataset one row at a time, every 2 seconds. The dashboard updates automatically — you do not need to click anything.

This is a simulation, not a real 5G connection. In a real deployment, you would connect to the Near-RT RIC via an E2 interface and receive live measurements. Implementing that requires physical radio hardware, which is beyond undergraduate project scope.

4.2 The complete data flow — step by step

Here is exactly what happens every 2 seconds during live monitoring:

8. The frontend (Next.js browser page) calls GET /api/stream/next via the Axios HTTP client.
9. The Next.js server proxies this to http://localhost:8000/api/stream/next (the FastAPI backend).
10. The FastAPI stream handler reads df.iloc[cursor] — the current row from the labeled DataFrame.
11. The cursor advances: cursor = (cursor + 1) % total_rows. At row 19,053,568, it wraps back to 0.
12. The 18 feature values are assembled into a named pandas DataFrame (1 row × 18 columns).
13. model.predict(X) returns 0 (Normal) or 1 (Degraded).
14. model.predict_proba(X) returns [prob_normal, prob_degraded]. We take index [1] for degradation probability.
15. A recommendation string is generated based on risk_code and probability.
16. The backend returns a StreamRow JSON: all 18 KPI values + actual_risk + predicted_risk + probability + recommendation.
17. React state updates: currentRow = response. The history array gets the new row appended (keeps last 60).
18. All dashboard elements re-render: KPI cards, line chart, probability gauge, alert box, session stats, ground truth panel.
19. setInterval fires again after 2000 ms. Go to step 1.

4.3 The stream state — how the server remembers its position

The backend uses a simple StreamState class:

```
class StreamState:
    df: DataFrame # the entire labeled CSV loaded into RAM at startup
    cursor: int    # index of the next row to serve
    total: int     # total number of rows (19 million+)
```

When the server starts, it loads the entire labeled_dataset.csv into a pandas DataFrame and stores it in memory. After that, every /stream/next call reads one row with df.iloc[cursor] — an instant lookup, no disk access.

The cursor wraps at the end: $\text{cursor} = (\text{cursor} + 1) \% \text{total}$. So after the last row, it goes back to row 0 and starts again. The live feed loops indefinitely.

4.4 What each element on the live monitoring screen shows

Screen Element	What it shows	Updates every
Start / Pause button	Starts or stops the 2-second polling loop	On click
Reset button	Rewinds CSV cursor to row 0, clears all history	On click
Session stats (top)	Total rows seen, Normal count, Degraded count and percentage	2 seconds
Dataset progress bar	Current position in the CSV (e.g. row 42 of 19,053,569)	2 seconds
8 KPI cards	Current values with colour coding: sky=healthy, amber=borderline, red=degraded	2 seconds
Live chart — blue line	Downlink throughput (Mbps) — last 30 readings, left Y-axis	2 seconds
Live chart — red line	Uplink error rate (%) — last 30 readings, right Y-axis	2 seconds
Probability gauge	Arc fills green→amber→red based on degradation probability	2 seconds
NORMAL / DEGRADED label	The model's binary verdict in large text	2 seconds
Alert box	Colour-coded (green/red) with recommendation text	2 seconds
Ground truth panel	Actual label vs predicted label. Shows ✓ or X for each row.	2 seconds

4.5 The manual prediction dashboard (home page /)

The home page lets you manually enter 18 KPI values and choose a model. Use the preset buttons for a quick demo.

Preset buttons

- "Load Healthy Sample" — fills all 18 fields with typical normal-operating values. Click Predict to see a green Normal result.

- "Load Degraded Sample" — fills values including 100% error rate and low CQI. Click Predict to see a red HIGH RISK result.

Model selector

- Random Forest (sky blue) — fastest, most explainable
- XGBoost (violet) — most accurate
- LSTM (emerald) — experimental, lower accuracy on this dataset

Switching model clears the current prediction so results from different models are not confused.

Predict button flow

20. Click "Predict Network Risk".
21. Axios sends POST /api/predict?model=xgboost with the 18 KPI values as JSON.
22. FastAPI validates the request (Pydantic checks all 18 fields are present and correct type).
23. The model produces a risk_code (0 or 1) and probability (0.0–1.0).
24. A recommendation is generated based on both values.
25. The dashboard shows: probability gauge, NORMAL/DEGRADED label, and alert box with recommendation.

5. API Parameters — Every Endpoint Explained

5.1 POST /api/predict

The most important endpoint. Accepts 18 KPI values as JSON and returns a prediction.

Query parameter

Parameter	Values	Default	Example URL
model	random_forest, xgboost, lstm	random_forest	/api/predict?model=xgboost

Request body — all 18 fields required

Field	Type	Description	Example
dl_mcs	float	Downlink Modulation and Coding Scheme (0–28). Higher = better channel.	9.6
dl_n_samples	int	Number of downlink transmission samples in this window.	147
dl_buffer_bytes	int	Bytes of downlink data waiting to be sent (queue backlog).	0
tx_brake_downlink_mbps	float	Actual downlink throughput in Megabits per second.	0.115
tx_pkts_downlink	int	Downlink packets sent in this window.	42
dl_cqi	float	Channel Quality Indicator reported by UE (0–15). Lower = worse.	7.0
ul_mcs	float	Uplink Modulation and Coding Scheme (0–28).	0.0
ul_n_samples	int	Number of uplink transmission samples in this window.	0
ul_buffer_bytes	int	Bytes of uplink data waiting to be sent.	0
rx_brake_uplink_mbps	float	Actual uplink throughput in Megabits per second.	0.0
rx_pkts_uplink	int	Uplink packets received by the base station.	0
rx_errors_uplink_pct	float	Uplink packet error rate as a percentage (0–100).	0.0
ul_sinr	float	Signal-to-Interference-plus-Noise Ratio in dB. 0 when UE idle.	0.0
phr	int	Power Headroom Report — remaining transmit power the UE has.	0
sum_requested_prbs	int	Physical Resource Blocks the UE requested this window.	790

Field	Type	Description	Example
sum_granted_prbs	int	Physical Resource Blocks the scheduler actually granted.	174
ul_turbo_iters	float	Average turbo decoder iterations per packet. 0 when UE idle.	0.0
prb_grant_ratio	float	Derived: $\text{sum_granted} / (\text{sum_requested} + 1)$. 0.22 = 22% of demand met.	0.22

Response fields

Field	Type	Description	Example
risk_label	string	"Normal" or "Degraded"	"Degraded"
risk_code	int	0 = Normal, 1 = Degraded	1
probability	float	Model confidence that network IS degraded (0.0–1.0)	0.96
recommendation	string	Plain-English advice based on risk and probability	"HIGH RISK: Immediate..."
model_used	string	Which model produced this prediction	"XGBoost"

Error codes

Code	Meaning	Fix
400	Unknown model name in ?model= parameter	Use random_forest, xgboost, or lstm
422	Missing or wrong-type field in request body	Check all 18 fields are present with correct types
503	Requested model not loaded	Run the training script for that model first
500	Unexpected error during prediction	Check server logs for details

5.2 GET /api/stream/next

Returns the next row from the labeled CSV and the model's prediction for it. Advances the cursor by 1.

Response field	Type	Description
row_index	int	Current position in the dataset (0 to total_rows-1)
timestamp	string	Original recording timestamp from the testbed CSV
total_rows	int	Total rows in the dataset (19,053,569 with your data)

Response field	Type	Description
[18 KPI fields]	float/int	All 18 feature values for this row
actual_risk	int	Ground truth label from label_dataset.py (0 or 1)
predicted_risk	int	Model's prediction for this row (0 or 1)
probability	float	Model confidence of degradation (0.0–1.0)
recommendation	string	Recommendation text

5.3 GET /api/health

Checks that the server is running and all models are loaded. Used by the dashboard header to show the green/red status indicator.

Response field	Description	Example
status	"ok" or "error"	"ok"
model_loaded	True if at least one model is loaded	true
model_name	Names of loaded models	"RF + XGB + LSTM"
message	Per-model load status	"Loaded: RF=yes XGB=yes LSTM=yes"

5.4 GET /api/comparison

Returns the comparison_report.json file containing training metrics for all three models. Used by the Model Comparison table at the bottom of the home page.

5.5 GET /api/stream/reset

Rewinds the CSV cursor to row 0. Used by the Reset button on the live monitoring page.

5.6 GET /api/stream/status

Returns the current cursor position without advancing it. Used for debugging.

6. Understanding the Evaluation Metrics

Every ML model is evaluated using the same four metrics. Understanding all four is essential for the viva.

6.1 The Confusion Matrix

A confusion matrix shows the four possible outcomes for a binary classifier:

	Predicted Normal	Predicted Degraded
Actual Normal (0)	True Negative (TN) ✓	False Positive (FP) ✗
Actual Degraded (1)	False Negative (FN) ✗	True Positive (TP) ✓

- TN (True Negative) — model said Normal, it really was Normal. Correct.
- FP (False Positive) — model said Degraded, it was actually Normal. False alarm.
- FN (False Negative) — model said Normal, it was actually Degraded. WORST outcome — a missed event.
- TP (True Positive) — model said Degraded, it really was Degraded. Correct.

False Negatives are the worst outcome in network monitoring. A missed degradation event means users are suffering while the system says everything is fine.

6.2 The Four Metrics Explained Simply

Metric	Formula	Question it answers	Why it matters here
Accuracy	$(TP+TN) / \text{Total}$	What % of ALL predictions were correct?	Good overall measure but misleading when 69% of rows are Normal
Precision	$TP / (TP+FP)$	Of all rows predicted Degraded, how many truly were?	High precision = few false alarms. Network teams trust alerts more.
Recall	$TP / (TP+FN)$	Of all actual Degraded rows, how many did we catch?	High recall = few missed events. Critical for network reliability.
F1 Score	$2 \times (P \times R) / (P + R)$	Balanced measure combining Precision and Recall	Single number for comparison — best metric to lead with in viva

6.3 Why all three models have 100% Precision

Precision is 100% for RF, XGBoost, and LSTM. This means zero false alarms — every time any model predicted Degraded, it was actually Degraded.

This happens because the labelling thresholds are conservative. A row needs two or more simultaneous bad KPI readings to be labelled Degraded. The models learn this conservative boundary and are reluctant to predict Degraded unless the evidence is strong.

The trade-off is lower Recall — some real degradation events are missed because they only triggered one bad indicator.

6.4 Three-way results summary

Metric	Random Forest	XGBoost	LSTM	Winner
Accuracy	98.56%	99.52%	77.27%	XGBoost
Precision	100.00%	100.00%	100.00%	All tied
Recall	94.87%	98.29%	18.80%	XGBoost
F1 Score	97.37%	99.14%	31.65%	XGBoost
Missed events (FN)	6 of 117	2 of 117	95 of 117	XGBoost
False alarms (FP)	0	0	0	All tied

7. Key Viva Questions — with Suggested Answers

Q: Does this project connect to a real 5G network?

No — but it uses real 5G data. The dataset was collected from a real srsRAN software-defined radio testbed. In the project, we replay the CSV row by row to simulate a live feed. A real E2 interface connection to a Near-RT RIC would require physical radio hardware and is listed as future work.

Q: Why did `clean_dataset.py` not drop columns like `IMSI` and `slice_id` in the multi-file version?

With a single recording session, `IMSI` and `slice_id` never changed — the zero-variance check dropped them. With many recording sessions, different experiments used different settings, so these columns now have multiple values and the zero-variance check lets them through. We fixed this by adding a `COLUMNS_TO_ALWAYS_DROP` hardcoded list that removes known identity and config columns regardless of how many files are merged.

Q: Why did the original LSTM crash with a memory error?

The original script tried to pre-compute all 19 million sliding-window sequences at once: $19\text{M} \times 20 \text{ steps} \times 18 \text{ features} \times 4 \text{ bytes} = 27 \text{ GB}$. No standard PC has that much RAM. We rewrote it using a Keras Sequence generator that creates one batch of 512 windows at a time. Memory usage stays around 750 KB per batch regardless of dataset size.

Q: Why is LSTM's recall so low with 19 million rows?

More rows did not help because the window size is still only 20 rows covering 5 seconds. With 19 million rows from many sessions, the data is richer overall, but the temporal window each prediction sees is still just 5 seconds of history. LSTM needs windows spanning minutes or hours to learn gradual degradation trends. The data quantity helps, but the window size is the binding constraint.

Q: What is the difference between `actual_risk` and `predicted_risk` in the stream endpoint?

`actual_risk` is the ground truth label that `label_dataset.py` assigned during data preparation — based on the quantile scoring rules. `predicted_risk` is what the currently loaded model computes fresh from the 18 KPI values when the stream/next request arrives. They usually match, but when they differ, the ground truth panel shows a `X Mismatch` — these are the false negatives and false positives.

Q: What would you do differently with more time?

Three things. First, increase the LSTM window size to cover minutes rather than seconds — this would give it enough temporal context to learn trend-based patterns. Second, add SHAP (SHapley Additive exPlanations) to explain individual predictions: not just "Degraded" but "because `rx_errors_uplink_pct` was in the top 10% and `dl_cqi` was in the bottom 25%." Third, implement a rolling buffer in the API so LSTM receives genuine recent history rather than a replicated single row.

8. Quick Reference — Numbers to Know for Your Viva

Fact	Value
Total rows after multi-file merge	~19 million
Original raw columns per CSV	36
Columns after cleaning	19 (18 features + 1 derived)
Identity/config columns always dropped	13 (IMSI, RNTI, slice_id, etc.)
Normal rows (approx)	71%
Degraded rows (approx)	29%
Train/test split	80% train / 20% test — no shuffle
LSTM window size	20 rows = 5 seconds at 250ms sampling
LSTM parameters	7,073 total
LSTM batch size (generator)	512 sequences = ~750 KB RAM
Random Forest trees	100
XGBoost boosting rounds	100
RF F1 Score (small dataset)	97.37%
XGBoost F1 Score (small dataset)	99.14%
LSTM F1 Score (small dataset)	31.65%
API endpoints	6
Live stream interval	2 seconds
Stream cursor wraparound	At total_rows — loops to 0 automatically
Feature importance #1	dl_mcs (22.97%) — modulation quality
Feature importance #5 (engineered)	prb_grant_ratio (6.56%) — congestion indicator